RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE

École Supérieure en Sciences et Technologies de l'Informatique et du Numérique

VISIONX

PLAN COMPLET DE CONDUITE DU PROJET

Agent d'automatisation métier — Marketing Automation

Équipe	VisionX — 6 membres
Chef d'équipe	Bouharizi Mohamed Anes
Membres	Sebti Iyed · Zouane Akram · Fellah Baha · Bertil Anis · Bahi Adem
Module	Conduite de Projet
Niveau	2ème année CS
Année universitaire	2026–2027

Document de planification et d'organisation
Version de travail — à valider avec les contrats de l'enseignant

Sommaire

- Introduction
- 1. Compréhension du projet
- 2. Architecture cible
- 3. Plan de travail par phases
- 4. Organisation de l'équipe
- 5. Livrables officiels L1 à L8
- 6. WBS — Découpage du travail
- 7. Planning et jalons
- 8. Risques et mesures préventives
- 9. Critères de validation
- 10. Premières actions à réaliser

Introduction

Ce document organise le travail de l'équipe VisionX pour le mini-projet de conception et développement d'agents logiciels intégrés à une plateforme commune. L'équipe a choisi un agent d'automatisation métier orienté marketing. Le plan couvre l'analyse, l'architecture, la conception, le développement, l'intégration au noyau, les tests, la documentation et la démonstration finale.

Limite documentaire : les documents contractuels AGENT_INTEGRATION_CONTRACT.doc, INTER_AGENT_DATA_CONTRACT.doc et DEVELOPER_INSTRUCTIONS.doc n'ont pas encore été fournis intégralement. Les détails exacts devront être confirmés à leur lecture. Les propositions de ce plan ne remplacent jamais les contrats officiels.

1. Compréhension du projet

1.1 Objectif général

Développer un seul agent logiciel métier destiné à être intégré dans le noyau commun fourni par l'enseignant. Le travail porte sur l'agent de l'équipe, et non sur la reconstruction du noyau (authentification, gestion des utilisateurs/entreprises, catalogue d'agents et passerelle d'intégration).

1.2 Choix de l'équipe

- Filière : agent d'automatisation métier.
- Agent choisi : Marketing Automation.
- Fonction indiquée : campagnes emailing et réseaux sociaux automatisées.
- Technologies indiquées : Python, Mautic, Postfix (SMTP).
- Produit publié : prospect.
- Objet consommé : commande.

1.3 Règles impératives

- Chaque équipe développe un seul agent ; ne jamais consulter ou copier le code d'une autre équipe.
- Les communications entre agents passent exclusivement par le noyau.
- Le contrat d'intégration est obligatoire et non négociable.
- Les objets métier doivent respecter le registre officiel.

- Les données sont isolées par utilisateur/entreprise.
- L'agent est livré sous forme d'image Docker fonctionnelle et autonome.
- Utiliser le dépôt privé fourni ; ajouter l'enseignant et l'assistant comme collaborateurs.
- Ne jamais rendre le dépôt public ni inviter une autre équipe.
- Les commits réguliers font partie de l'évaluation.

1.4 Périmètre fonctionnel initial

1. Gestion et mise à jour des prospects.
2. Création et gestion des campagnes marketing.
3. Préparation de contenus ou modèles d'emails.
4. Déclenchement ou planification des campagnes.
5. Envoi email via le service retenu et configuré.
6. Publication des objets prospect.
7. Consommation des objets commande.
8. Suivi des états et résultats réellement disponibles.

Les fonctions exactes concernant les réseaux sociaux, les ouvertures/clics, la segmentation et les automatisations avancées restent à arbitrer par l'équipe et à valider selon les outils disponibles.

2. Architecture cible

2.1 Principe

L'agent Marketing Automation est un service autonome qui s'intègre au noyau par les interfaces contractuelles. Il ne se connecte jamais directement au code ou à la base de données d'un autre agent. Les échanges d'objets métier passent par le noyau.

2.2 Architecture logique proposée

Couche	Composants / responsabilité
Noyau commun	Authentification, tenants, catalogue d'agents et passerelle d'intégration (fourni par l'enseignant).
Contrat	Endpoints contractuels, échanges d'objets métier et notifications via le noyau.
Agent VisionX	API Layer, Core Services, Prospect Manager, Campaign Manager, Automation Engine, Integration Service.
Persistance	PostgreSQL proposé pour les données propres à l'agent ; séparation stricte par tenant.
Outils marketing	Mautic et Postfix/SMTP, indiqués dans le sujet ; configuration à confirmer.
Packaging	Docker pour une exécution autonome et reproductible.

2.3 Responsabilités des composants

Composant	Responsabilité proposée
API Layer	Recevoir les requêtes, valider les données et exposer les interfaces nécessaires.
Core Services	Appliquer les règles métier et coordonner les modules.
Prospect Manager	Créer, modifier, consulter et organiser les prospects.
Campaign Manager	Créer les campagnes, gérer leur état et leur contenu.
Automation Engine	Exécuter les déclencheurs et actions planifiées.

Composant	Responsabilité proposée
Integration Service	Publier les prospects et traiter les commandes selon le contrat du noyau.
Base de données	Persister les données propres à l'agent avec séparation par tenant.
Mautic / Postfix	Fournir les capacités marketing et email selon la configuration retenue.
Docker	Fournir un packaging autonome et reproductible.

2.4 Flux métier inter-agents

Publication d'un prospect : création/mise à jour dans l'agent → validation métier → publication via le mécanisme du noyau → notification des agents abonnés selon leurs déclarations.

Consommation d'une commande : commande publiée dans le noyau → notification/réception par l'agent → validation du schéma et du tenant → déclenchement éventuel d'une action marketing autorisée. La règle exacte doit être définie par l'équipe ; ne pas supposer un envoi automatique.

2.5 Technologies

Élément	Technologie	Statut
Langage	Python	Indiqué dans le sujet.
Marketing automation	Mautic	Indiqué ; version/configuration à confirmer.
Email	Postfix (SMTP)	Indiqué ; configuration à confirmer.
Base de données	PostgreSQL	Proposition de l'équipe, à valider.
Framework API	À choisir	Non fixé dans les pages fournies ; dépend du contrat.
Conteneurisation	Docker	Exigence explicite.
Versionnement	Dépôt GitHub privé fourni	Exigence explicite.

3. Plan de travail par phases

Phase 1 — Analyse des contrats et architecture

1. Lire intégralement AGENT_INTEGRATION_CONTRACT.doc.
2. Lire intégralement INTER_AGENT_DATA_CONTRACT.doc.
3. Lire DEVELOPER_INSTRUCTIONS.doc.
4. Extraire endpoints, méthodes HTTP, paramètres, schémas et erreurs imposés.
5. Identifier authentification, activation, désactivation et statut.
6. Confirmer les schémas prospect et commande dans le registre.
7. Définir les cas d'utilisation et le périmètre MVP.
8. Produire les diagrammes d'architecture, cas d'utilisation, séquence et données.
9. Choisir et justifier les technologies non imposées.
10. Faire valider l'architecture par toute l'équipe.

Résultat attendu : Dossier d'architecture, matrice des exigences, diagrammes et décisions techniques.

Phase 2 — Conception fonctionnelle et technique

1. Décrire les acteurs et leurs permissions.
2. Décrire les scénarios de création de prospect, segmentation, campagne, lancement, arrêt et suivi.
3. Définir les entités et relations de la base de données.

4. Définir les API métier internes, validations et règles de gestion.
5. Définir les événements déclencheurs et le traitement des commandes.
6. Définir sécurité, journalisation et isolation par tenant.
7. Préparer les critères d'acceptation.

Résultat attendu : Spécifications, modèle de données, contrat API interne et backlog.

Phase 3 — Initialisation de l'environnement

1. Récupérer le dépôt privé fourni par l'enseignant.
2. Ajouter l'enseignant et l'assistant comme collaborateurs.
3. Vérifier que le dépôt reste privé et n'invite aucune autre équipe.
4. Créer les branches et règles de revue de code.
5. Préparer Python, environnement virtuel et dépendances.
6. Préparer Docker et les fichiers de configuration.
7. Préparer PostgreSQL si le choix est confirmé.
8. Installer/configurer Mautic et Postfix selon les contraintes disponibles.
9. Préparer .env.example et la documentation d'installation.

Résultat attendu : Environnement de développement reproductible.

Phase 4 — Développement du backend

1. Initialiser l'application et sa configuration.
2. Mettre en place la connexion à la base de données.
3. Créer les modèles et migrations.
4. Implémenter les services de gestion des prospects.
5. Implémenter les services de gestion des campagnes.
6. Implémenter les listes et segments si retenus.
7. Implémenter les API et validations.
8. Ajouter gestion des erreurs et journalisation.
9. Vérifier l'isolation des données par utilisateur/entreprise.

Résultat attendu : Backend métier testable.

Phase 5 — Fonctionnalités Marketing Automation

1. Créer et modifier les prospects.
2. Importer/enregistrer les prospects selon les sources autorisées.
3. Organiser les prospects en listes ou segments.
4. Créer et configurer une campagne.
5. Préparer les modèles et contenus marketing.
6. Configurer l'envoi via Mautic/Postfix selon l'architecture retenue.
7. Définir les déclencheurs et règles d'automatisation.
8. Traiter les événements de commande selon les règles métier décidées.
9. Suivre les statuts et résultats réellement disponibles.

Résultat attendu : Scénario marketing de bout en bout fonctionnel.

Phase 6 — Intégration au noyau

1. Implémenter /health, /tenant/activate, /tenant/deactivate, /tenant/{id}/status et /config-schema selon le contrat.
2. Respecter l'enveloppe standard {status, data, error}.
3. Créer agent.manifest.json conforme au schéma officiel.
4. Déclarer prospect comme objet produit et commande comme objet consommé après vérification du registre.
5. Implémenter la publication du prospect.
6. Implémenter la réception/notification et le traitement des commandes.
7. Implémenter /exchange/notify si requis pour l'agent consommateur.
8. Gérer erreurs, doublons et événements répétés selon le contrat.
9. Tester les échanges avec le noyau ou son environnement de test.

Résultat attendu : Agent intégrable au noyau commun.

Phase 7 — Tests et sécurité

1. Tests unitaires des services.
2. Tests des endpoints et formats de réponse.
3. Tests de validation des objets métier.
4. Tests de publication d'un prospect et consommation d'une commande.
5. Tests d'isolation entre tenants.
6. Tests d'activation, désactivation et statut.
7. Tests des erreurs réseau et services externes.
8. Tests de lancement et arrêt Docker.
9. Correction et traçabilité des anomalies.

Résultat attendu : Version stable, sécurisée et démontrable.

Phase 8 — Packaging et déploiement

1. Finaliser Dockerfile et Docker Compose si nécessaire.
2. Séparer configuration et secrets du code.
3. Tester la construction de l'image et le démarrage sur environnement propre.
4. Vérifier health checks et logs.
5. Compléter README : prérequis, configuration, lancement, tests et dépannage.
6. Vérifier le dépôt avant T6.

Résultat attendu : Image Docker autonome et instructions reproductibles.

Phase 9 — Suivi et bilan

1. Définir des KPI adaptés au périmètre réellement implémenté.
2. Comparer prévisions et réalisations.
3. Suivre tâches terminées, en retard et restantes.
4. Documenter écarts, risques et décisions.
5. Évaluer la conformité au contrat et au registre.
6. Rédiger réalisations, limites, difficultés et perspectives.

Résultat attendu : Livrables L6 et L7 cohérents avec le projet réel.

Phase 10 — Présentation finale

1. Construire un support PowerPoint clair.
2. Présenter besoin, objectif, architecture et technologies.
3. Démontrer un scénario complet : prospect, campagne et échange via le noyau.
4. Montrer l'agent actif et fonctionnel sur la plateforme.
5. Présenter les preuves de tests et répondre aux questions.
6. Répéter la démonstration et prévoir un scénario de secours.

Résultat attendu : Présentation L8 avec démonstration live.

4. Organisation de l'équipe

Répartition initiale proposée ; elle doit être confirmée collectivement. Tous les membres doivent comprendre l'architecture globale et participer aux revues et tests.

Membre	Rôle proposé	Responsabilités principales
Bouharizi Mohamed Anes	Chef de projet / architecture	Coordination, architecture, contrats, intégration globale et arbitrages.
Sebti Iyed	Backend / API	Structure backend, endpoints métier, validations et documentation API.
Zouane Akram	Données / prospects	Modèle de données, persistance et gestion des prospects.
Fellah Baha	Campagnes / automatisation	Logique des campagnes, segmentation et scénarios.
Bertil Anis	Outils marketing / email	Étude et intégration Mautic et Postfix/SMTP.
Bahi Adem	Tests / sécurité / Docker	Tests, isolation des tenants, packaging et déploiement.

Règles de collaboration

- Chaque tâche possède un responsable et un critère d'acceptation.
- Les interfaces entre modules sont définies avant le développement parallèle.
- Les changements importants sont relus par un autre membre.
- Les commits sont réguliers, explicites et associés à une tâche.
- Les secrets et clés SMTP ne sont jamais commités.
- Les décisions d'architecture sont consignées.
- Le code reste dans le dépôt privé de l'équipe.

5. Livrables officiels L1 à L8

Liv.	Contenu demandé	Volume / échéance	Nom de fichier
L1	Orientation : agent, objectif, justification, méthodologie, plan, sources et risques.	~1 page Word, avant T2	METIER_DO_NomAgent.docx
L2	Charte : définition, objectifs, bénéfices, livrables, tâches, ressources, affectation et risques.	6–7 pages, avant T3	METIER_Charte_NomAgent.docx
L3	PV de lancement : rôles et engagement collectif sur le contrat.	1–2 pages, avant T4	METIER_PVLancement_NomAgent.docx
L4	WBS/OBS/RACI, échéancier, jalons, dépendances, PERT, Gantt, coûts et efforts.	5–6 pages, avant T5	METIER_PLAN_NomAgent.docx

Liv.	Contenu demandé	Volume / échéance	Nom de fichier
L5	Rapport d'exécution, versions, problèmes, conformité, code complet et dépôt privé.	6–7 pages hors annexes, avant T6	METIER_RAP_NomAgent.docx + GitHub
L6	Suivi : KPI prévisionnel/réalisé et conformité au contrat/registre.	3–5 pages hors annexes, avant T7	METIER_Suivi_NomAgent.docx
L7	Bilan : réalisations, écarts, perspectives et leçons apprises.	4–5 pages hors annexes, avant T8	METIER_Bilan_NomAgent.docx
L8	Présentation orale 20 min + 5 min questions, démonstration live.	PowerPoint uniquement, avant T9	METIER_PRES_NomAgent.pptx

Les noms doivent suivre exactement la convention de l'enseignant ; remplacer NomAgent par le nom retenu.

6. WBS — Découpage du travail

Code	Lot	Activités / sorties
1.0	Pilotage	Organisation, réunions, suivi, risques et décisions.
1.1	Analyse des contrats	Lecture, extraction des exigences, matrice de conformité.
1.2	Architecture	Diagrammes, composants, flux et choix techniques.
2.0	Conception	Cas d'utilisation, modèle de données, API, critères d'acceptation.
3.0	Environnement	Dépôt privé, Git, Python, Docker, base et outils.
4.0	Backend	Modèles, services, API, validation et logs.
5.0	Métier marketing	Prospects, campagnes, segmentation et automatisations.
6.0	Intégration noyau	Manifeste, endpoints contractuels, publication/consommation.
7.0	Qualité	Tests unitaires, intégration, sécurité et corrections.
8.0	Livraison technique	Image Docker, README, vérification d'installation.
9.0	Documentation	L5, L6, L7 et preuves de conformité.
10.0	Présentation	Slides, scénario live, répétition et L8.

7. Planning et jalons

Les pages fournies indiquent les échéances relatives T2 à T9, mais pas les dates calendaires exactes. Il faut positionner ce planning sur le calendrier réel du cours.

Échéance	Priorité	Jalon	Condition de sortie
Avant T2	Orientation	L1	Agent et périmètre justifiés.
Avant T3	Charte	L2	Objectifs, tâches, ressources et risques validés.
Avant T4	Lancement	L3	Rôles et engagement confirmés.
Avant T5	Planification	L4	WBS, OBS, RACI, Gantt et dépendances.
Avant T6	Développement	L5	Code, dépôt privé, Docker et rapport.
Avant T7	Suivi	L6	KPI et conformité documentés.
Avant T8	Clôture	L7	Bilan argumenté.
Avant T9	Présentation	L8	Démonstration live fonctionnelle.

Dépendances critiques

- L'architecture dépend de la lecture des trois documents techniques.
- Le modèle des objets dépend du registre officiel inter-agents.
- L'intégration dépend du respect du contrat d'intégration.
- Les tests d'échange dépendent de la disponibilité du noyau ou d'un environnement de test.
- La démonstration dépend d'une image Docker et d'un agent activé sur la plateforme.

8. Risques et mesures préventives

Risque	Impact possible	Prévention / réponse
Contrats mal compris	Agent non intégrable et perte de points.	Lire les documents, créer une matrice de conformité et valider tôt.
Objet métier mal défini	Échec des échanges.	Utiliser exclusivement les schémas du registre officiel.
Retard de développement	Livrables incomplets.	Découper en tâches courtes et suivre chaque semaine.
Dépendance Mautic/SMTP	Envoi email indisponible.	Tester tôt et prévoir un mode de test sans envoi réel.
Fuite entre tenants	Non-conformité et risque de sécurité.	Tester l'isolation et contrôler chaque requête.
Problème Docker	Agent impossible à lancer.	Construire et tester l'image régulièrement.
Perte de code / conflits	Retard et mauvaise collaboration.	Dépôt privé officiel, commits fréquents et revue.
Démo indisponible	Présentation finale affaiblie.	Répéter et préparer données/scénario de secours.

9. Critères de validation

- Le périmètre métier est défini et documenté.
- Les contrats officiels sont lus et la conformité est tracée.
- L'agent expose tous les endpoints obligatoires.
- Le manifeste respecte le schéma imposé.
- L'agent publie correctement les prospects.
- L'agent consomme correctement les commandes.
- Les échanges passent par le noyau.
- Les données sont isolées par utilisateur/entreprise.
- Les tests essentiels sont exécutés et documentés.
- L'image Docker est fonctionnelle et autonome.
- Le dépôt privé contient code, manifeste, Dockerfile et README.
- Les livrables L1 à L8 sont préparés dans les délais.
- La démonstration live fonctionne sur la plateforme commune.

10. Premières actions à réaliser

1. Obtenir les trois documents contractuels officiels.
2. Les lire en réunion et attribuer un responsable à chaque partie.
3. Créer une matrice des exigences : exigence, source, module, responsable, test et statut.

4 Confirmer les schémas exacts de prospect et commande.

5 Définir le MVP : fonctionnalités indispensables à la première démonstration.

6 Valider l'architecture et les technologies.

7 Produire L1, puis préparer L2 et L4 selon les échéances.

Conclusion

Ce plan donne à VisionX une démarche structurée pour réaliser l'agent Marketing Automation depuis l'analyse jusqu'à la présentation finale. La priorité immédiate est l'analyse des contrats et la validation de l'architecture. Les choix techniques proposés doivent être confirmés avant implémentation. La réussite dépend du respect du contrat d'intégration, du registre des objets métier, de l'isolation des données et du packaging Docker exigé.

À retenir : ne commencez pas le développement des interfaces contractuelles ou des échanges métier avant d'avoir lu les trois documents de référence. Leurs schémas et exigences priment sur les propositions de ce plan.